PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

Similarity and Distance 13

Centroid Clustering 27

Hierarchical Clustering 41

Index 53

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Similarity and Distance

Inside the Feature Space

The Why

In supervised learning[1], the learning process relies on data samples x paired with their ground truth labels $\tilde{y}$, which allows a model θ to learn a mapping,

$$\begin{aligned}\theta(x) &= \hat{y}, \\ &\approx \tilde{y}.\end{aligned}$$

[1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org

On the concept of truth, $\tilde{y}$, while being called ground truth, is often an approximation of the true label y. It may come from human annotation, measurements, or other sources.

The presence of labels provides a clear signal for learning, as it allows optimization of a loss function that measures the discrepancy between predictions and true labels. The loss then guides the learning process, enabling the model to adjust its parameters along the gradient to minimize this discrepancy:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\hat{y}, \tilde{y}).$$

Facing the absence of labels, we are left with only the data samples[2] x and no explicit signal to guide learning. The methods we will explore in this course are designed to discover and leverage the inherent structure and patterns[3] in the data itself, without relying on external supervision, thus mostly referred to as unsupervised learning.

[2] J. Rowley. The wisdom hierarchy: Representations of the dikw hierarchy. *Journal of Information and Communication Science*, 33(2):163–180, 2007. doi: 10.1177/0165551506070706. Retrieved November 9, 2024

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

In a first step this requires us to,

- define what we mean by pattern in the context of data.
- understand the geometry of data, which includes concepts of distance, similarity, and neighborhood.
- learn how to engineer data representations that make pattern more accessible.

Knowledge Pyramid
Data (1) raw signal
→ Information (*banana*) endowed with meaning
→ Knowledge (≠ *coffee*) structured and contextualized information
→ Wisdom or *knowing and appreciating the Why*.

THE FUNDAMENTAL QUESTION in unsupervised learning is to find pattern, represent it, and utilize it.

Hands On Experience

CONSIDER six data points in two dimensions. Which points seem to belong together? Why do you think that? How many groups do you see? To which group would you assign new points? Draw areas around each group.

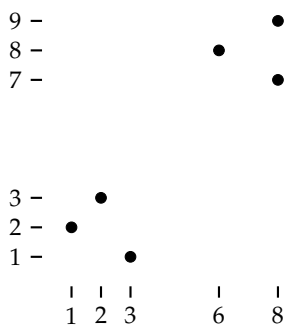


Figure 1: Six points in 2D. Can you see the two clusters? Or are there six? Or maybe one?

WITHOUT LABELS, pattern are ambiguous. With just a single additional point, the pattern can change drastically. Where does the new point belong? Does it belong to the first cluster, the second, or does it form its own cluster? Does this new point change the pattern of the original six points? How would you draw areas around the clusters now? How did that change your assumptions of the first pattern?

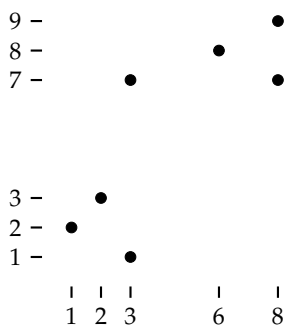


Figure 2: Seven points in 2D. The point at $(3, 7)$ creates ambiguity. Is it part of the first cluster, the second, or its own cluster?

HUMANS ARE REMARABLY GOOD AT FINDING PATTERN IN DATA, even with very little information, and even if it is completely off. It is a fundamental part of our cognition, and it is what allows us to make sense of the world. However, it also motivates the need for

PAREIDOLY is the tendency to perceive meaningful patterns in random data. Such as animals in clouds.

methods to quantify and formalize pattern. On that note it is also good practice to start with a hypothesis of what you expect to find in the data, and then test that hypothesis with the data, rather than just looking for any pattern that may emerge. And on a short managerial note, if the knowledge you will gain will not make you take action, then don't waste your time measuring it in the first place.

THE LEARNING OBJECTIVES of this lecture are that you will be able to:

- Understand the key role of distance and similarity in unsupervised learning and pattern recognition.
- Compute and interpret common distance metrics (Euclidean, Manhattan, Cosine, Mahalanobis).
- Apply appropriate normalization and scaling strategies and understand why they are necessary.

Distance and Similarity

QUITE INTUITIVELY, you already connected points and formed groups in terms of a similarity which was based on the closeness between data points. This concept will carry you very far.

A DISTANCE FUNCTION $d : X \times X \to \mathbb{R}_{\geq 0}$, which means taking two samples out of the set X, mapping them to a non-negative real number, must satisfy:

$$\text{Non-negativity:} \quad d(x,y) \geq 0 \tag{1}$$

$$\text{Identity:} \quad d(x,y) = 0 \iff x = y \tag{2}$$

$$\text{Symmetry:} \quad d(x,y) = d(y,x) \tag{3}$$

$$\text{Triangle inequality:} \quad d(x,z) \leq d(x,y) + d(y,z) \tag{4}$$

EUCLIDEAN DISTANCE is probably the most familiar distance metric, measuring the straight-line distance between two points in space.

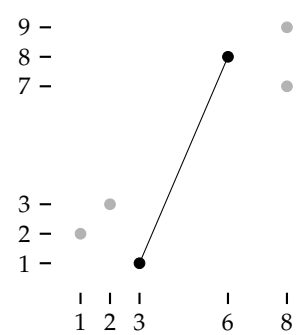


Figure 3: Euclidean distance between points (3,1) and (6,8).

And it was probably the one you implicitly used to connect the points in the previous exercise. It is defined as:

$$\begin{aligned} d_{\text{Euclidean}}(x,y) &= \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \\ &= \|x - y\|_2 \end{aligned}$$

Calculating the Euclidean distance between points $A = (3,1)$ and $B = (6,8)$:

$$\begin{aligned} d(A,B) &= \sqrt{(6-3)^2 + (8-1)^2} \\ &= \sqrt{9+49} \\ &= \sqrt{58} \qquad \approx 7.62 \end{aligned}$$

EUCLIDEAN DISTANCE assumes all dimensions are equally important and measured in the same units. It also assumes a continuous, smooth space. It is trivial to think of scenarios where these assumptions do not hold, such as when features have different scales (e.g. age vs. income) or when the features are not linearly related or not equaly important (e.g. size vs. IQ). And then there are the cases where the units are ambiguous for the task like distances on maps.

FEATURES can be thought of as dimensions in a vector space. Each data point is a vector in this space.

MANHATTAN DISTANCE is also called L_1 distance or taxicab distance. Instead of measuring the euclidean distance, it measures the axis-aligned path along grid lines:

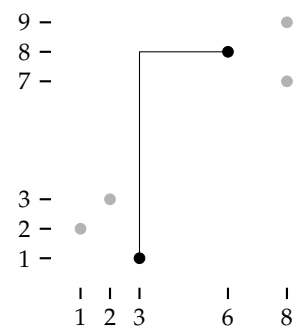


Figure 4: Manhattan (taxicab) distance between points (3,1) and (6,8): path follows grid lines.

$$d_{\text{Manhattan}}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
$$= \|x - y\|_1$$

Figure 5: Grid, 1811 (public domain) see Bridgeman Art Library v. Corel Corp.

ESPECIALLY USEFUL when movement is constrained to axes, such as for city blocks like the ones in Manhattan, pixel grids, or when you want robustness to outliers in individual dimensions.

Calculating the Manhattan distance between points $A = (3, 1)$ and $B = (6, 8)$:

$$d_M(A, B) = |6 - 3| + |8 - 1|$$
$$= 3 + 7$$
$$= 10$$

COSINE SIMILARITY measures the angle between two vectors, ignoring their magnitude. It is especially useful when the correlation of the features of the data matters more than its absolute values.

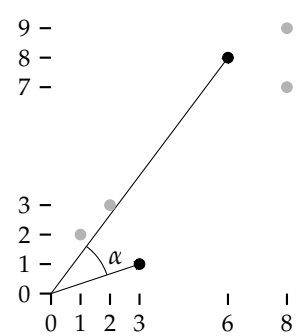


Figure 6: Cosine similarity between points (3,1) and (6,8): measuring the angle α between the position vectors.

When magnitude in the feature space does not matter only direction we use cosine similarity. It is defined as:

Calculating the cosine similarity between points $A = (3, 1)$ and $B = (6, 8)$:

$$\text{sim}(A, B) = \frac{3 \cdot 6 + 1 \cdot 8}{\sqrt{3^2+1^2}\,\sqrt{6^2+8^2}}$$
$$= \frac{26}{\sqrt{10} \cdot 10}$$
$$\approx 0.82$$

$$
\begin{aligned}
d_{\text{cosine}}(x, y) &= 1 - \frac{x \cdot y}{\|x\| \|y\|} \\
&= 1 - \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}
\end{aligned}
$$

The right-hand term is the cosine similarity, which ranges from -1 (opposite) to $+1$ (identical direction), so the distance ranges from 0 to 2.

THINK of a vector multiplication as a projection of one vector onto another.

MAGNITUDE IS IRRELEVANT for example, in text analysis, documents can have vectors of very different lengths as some documents have more words than others, but cosine similarity will still focuse on how many specific words per total word count. Similarly, in recommendation systems, users might have preference vectors with different scales, as some users rate more items, some less, but cosine similarity will compare the pattern of preferences.

MAHALANOBIS DISTANCE generalizes Euclidean distance by accounting for different variances and correlations among features. It measures distance in terms of the data's own covariance structure. In other words, it transforms the space so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods. The covariance matrix Σ captures the spread and correlation of the data, and is defined as:

$$
\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \mu_i)(x_i - \mu_i)^T
$$

where μ is the mean vector of the data.

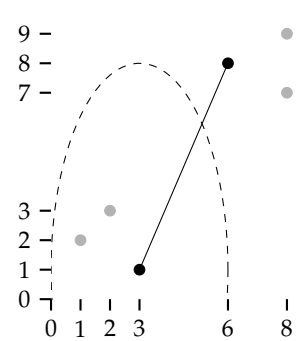


Figure 7: Mahalanobis distance between points (3,1) and (6,8): the dashed ellipse shows equidistant points under covariance Σ, replacing the Euclidean circle with an ellipse that reflects the data's spread.

Euclidean distance treats all dimensions equally, but real data often has different variances per dimension or correlations between

dimensions. The Mahalanobis distance compensates by weighting each dimension according to the data's spread. The inverse covariance matrix Σ^{-1} is used to weight the distance calculation, giving more weight to dimensions with less variance and less weight to dimensions with more variance.

$$d_{\text{Mahalanobis}}(x, y) = \sqrt{(x-y)^T \Sigma^{-1} (x-y)}$$

where Σ is the covariance matrix of the data.

Calculating the Mahalanobis distance between $A = (3, 1)$ and $B = (6, 8)$ with $\Sigma = \left(\begin{smallmatrix} 9 & 0 \\ 0 & 49 \end{smallmatrix}\right)$:

$$\begin{aligned} d_M(A, B) &= \sqrt{\frac{3^2}{9} + \frac{7^2}{49}} \\ &= \sqrt{1+1} \\ &= \sqrt{2} \\ &\approx 1.41 \end{aligned}$$

IT TRANSFORMS THE SPACE so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods.

DATA PREPROCESSING, before computing distances, ensures that features are comparable. You have already seen, how the Mahalanobis distance incorporates the covariance structure of the data to account for different scales and correlations. With the Euclidean distance, if features are on different scales, the distance will be dominated by the larger-scaled feature. If one feature's measure has a different magnitude than the other, the larger-scaled feature will dominate the distance calculation. Preprocessing brings all features to a common ground by transforming the feature space into a standardized scale.

NORMALIZATION typically refers to scaling features to a specific range, such as $[0, 1]$. While standardization refers to centering features to have mean 0 and scaling to have standard deviation 1.

Z-SCORE NORMALIZATION transforms each feature to have mean 0 and standard deviation 1. This centers the data at the origin and scales each dimension by its own spread:

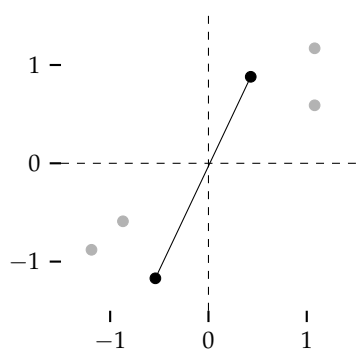


Figure 8: Our six points after z-score normalization: centered at the origin with unit variance in each dimension.

The z-score of each feature is computed as:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} \tag{5}$$

where μ_i and σ_i are the mean and standard deviation of feature i.

Normalizing point $A = (3, 1)$:

$$\begin{aligned} \mu &= (4.67,\ 5.00) \\ \sigma &= (3.08,\ 3.41) \\ z_x &= \frac{3 - 4.67}{3.08} \\ &= -0.54 \\ z_y &= \frac{1 - 5.00}{3.41} \\ &= -1.17 \\ A' &= (-0.54,\ -1.17) \end{aligned}$$

Use when features have different units or scales (e.g., age in years, vs. body height in milimiters). However, this is sensitive to outliers, as they influence μ and σ.

Min-Max Scaling transforms features to a fixed range $[0, 1]$. Each feature's minimum maps to 0 and its maximum to 1:

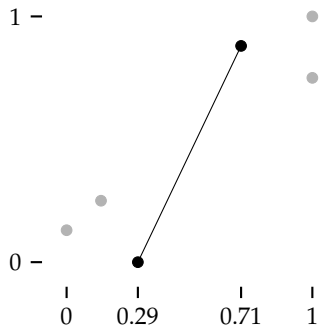


Figure 9: Our six points after min-max scaling: all values lie in $[0, 1]$.

$$x'_i = \frac{x_i - \min_i}{\max_i - \min_i} \tag{6}$$

Scaling point $A = (3, 1)$ with $x \in [1, 8]$, $y \in [1, 9]$:

$$\begin{aligned} x' &= \frac{3-1}{8-1} \\ &= \frac{2}{7} \\ &\approx 0.29 \\ y' &= \frac{1-1}{9-1} \\ &= 0 \\ A' &= (0.29,\ 0) \end{aligned}$$

Use when you need bounded values. Unlike z-score, min-max guarantees a fixed range but is even more sensitive to outliers since a single extreme value stretches the scale and compresses other ranges.

Choosing a Distance Metric encodes your assumptions about what makes two data points similar in a given context. The same pair of points can appear close or far apart depending on the metric. The choice should be driven by the structure of your data, the context you are in, and the question you are answering.

Distance between A=$(3, 1)$ and B=$(6, 8)$ under each metric:

$$\begin{aligned} d_{\text{Euclidean}} &= 7.62 \\ d_{\text{Manhattan}} &= 10 \\ d_{\text{Cosine}} &= 0.18 \\ d_{\text{Mahalanobis}} &= 1.41 \end{aligned}$$

Same points, four different answers. The distance defines the similarity.

A practical rule of thumb: try Euclidean first as a baseline, then ask whether the assumptions it makes, equal scales, independent features, magnitude matters, actually hold for your data. Each violation points toward a more suitable metric.

Examples & Exercises

Exercises are for practice and reinforcing concepts. Try to solve them on your own first, try things, play with it, discuss, this is not a time trial. And there is no shame in not ending up at the right answer, in the same sense, that uncovering great questions and tossing them around is usually pretty fruitful on the long run.

$A = (2, 5, 1)$
$B = (6, 1, 3)$
$C = (4, 5, 2)$

Computing by hand builds the intuition that no amount of library calls can replace. Step through the arithmetic slowly.

Given points A, B, and C, compute the Euclidean distance $d_E(A, B)$, the Manhattan distance $d_M(A, C)$, and the cosine similarity $\text{sim}(A, B)$ with its corresponding cosine distance. If you feel you need more practice, go for it.

Euclidean distance sums squared coordinate differences:

$$\begin{aligned} d_E(A, B) &= \sqrt{(6-2)^2 + (1-5)^2 + (3-1)^2} \\ &= \sqrt{16 + 16 + 4} \\ &= \sqrt{36} \\ &= 6 \end{aligned}$$

Manhattan distance sums absolute differences instead, no squaring, no root:

$$\begin{aligned} d_M(A, C) &= |2-4| + |5-5| + |1-2| \\ &= 2 + 0 + 1 \\ &= 3 \end{aligned}$$

Cosine similarity measures the angle between the two vectors, independent of their length:

$$\begin{aligned} \text{sim}(A, B) &= \frac{A \cdot B}{\|A\| \, \|B\|} \\ &= \frac{2{\cdot}6 + 5{\cdot}1 + 1{\cdot}3}{\sqrt{4{+}25{+}1}\,\sqrt{36{+}1{+}9}} \\ &= \frac{20}{\sqrt{30}\,\sqrt{46}} \\ &\approx 0.54 \end{aligned}$$

The cosine distance follows as:

$$\begin{aligned} d_{\cos} &= 1 - 0.54 \\ &= 0.46 \end{aligned}$$

The metric is a modelling choice. There is no formula that tells you which distance to use, it follows from understanding your data and the context you are in. In the following scenarios, decide which distance metric is most appropriate and justify your choice. First

reason geometrically, then commit to an answer. Zoom-in so you can not see the margin notes, and try to solve it on your own first. Then read the margin notes for the answer.

A NEWS PLATFORM represents 10 000 articles as word-frequency vectors. Some articles are 100 words long, others 5 000 words. You want to find articles that cover similar topics.

Cosine similarity. Article length inflates the magnitude of the frequency vector but does not change its direction. Cosine similarity ignores magnitude, comparing only the distribution of words.

A DELIVERY COMPANY plans routes through Manhattan's grid. Pickup locations are given as (street, avenue) coordinates.

Manhattan distance. Movement on a grid is constrained to horizontal and vertical steps. The shortest driving path between two intersections follows the L_1 norm, not the straight line.

A COFFEE QUALITY LAB measures sourness (0 to 200) and bitterness (0 to 500) for each batch.

Mahalanobis distance. The features live on different scales and are correlated. Mahalanobis accounts for both by using the covariance matrix, producing ellipsoidal contours that align with the data's spread.

GIVEN THREE COFFEE BATCHES measured by sourness and bitterness:

$$
\begin{aligned}
P_1 &= (80,\ 400) \\
P_2 &= (200,\ 160) \\
P_3 &= (110,\ 220)
\end{aligned}
$$

compute the sample covariance matrix and use it to find the Mahalanobis distance between P_1 and P_3.

The mean vector:

$$
\begin{aligned}
\mu_x &= 130 \\
\mu_y &= 260 \\
\mu &= (130,\ 260)
\end{aligned}
$$

Deviations from the mean:

$$
\begin{aligned}
P_1 - \mu &= (-50,\ \ 140) \\
P_2 - \mu &= (\ \ 70,\ -100) \\
P_3 - \mu &= (-20,\ -40)
\end{aligned}
$$

The sample covariance matrix sums all three outer products:

$$
\begin{aligned}
\Sigma &= \frac{1}{N-1} \sum (x_i - \mu)(x_i - \mu)^T \\
&= \frac{1}{2} \begin{pmatrix} 7800 & -13200 \\ -13200 & 31200 \end{pmatrix} \\
&= \begin{pmatrix} 3900 & -6600 \\ -6600 & 15600 \end{pmatrix}
\end{aligned}
$$

We invert Σ analytically using the 2×2 formula:

$$\begin{aligned}\det(\Sigma) &= 3900 \cdot 15600 - 6600^2 \\ &= 17{,}280{,}000 \\ \Sigma^{-1} &= \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}\end{aligned}$$

For a 2×2 matrix:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A^{-1} = \frac{1}{ad - bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Swap the diagonal, negate the off-diagonal, divide by the determinant.

The difference vector is $\delta = P_3 - P_1 = (30,\ -180)$. Applying Σ^{-1}:

$$\begin{aligned}\Sigma^{-1}\delta &= \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}\begin{pmatrix} 30 \\ -180 \end{pmatrix} \\ &= \frac{1}{17{,}280{,}000}\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix}\end{aligned}$$

INTUITIVELY, A^{-1} undoes the transformation A: if A stretches and rotates space, A^{-1} maps everything back, with $AA^{-1} = I$. For Σ, the inverse rescales each dimension by its variance and removes correlations, so distances are measured in standard-deviation units.

Taking the dot product with δ gives the squared distance:

$$\begin{aligned}d_M^2 &= \delta^T(\Sigma^{-1}\delta) \\ &= \frac{1}{17{,}280{,}000}(30,\ -180)\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix} \\ &= \frac{69{,}120{,}000}{17{,}280{,}000} \\ &= 4 \\ d_M &= \sqrt{4} \\ &= 2\end{aligned}$$

Compare with the Euclidean distance, which does not account for the different scales and correlation. The raw displacement of 30 sourness units and 180 bitterness units yields an overall distance of 182.5, making P_3 seem very far from P_1, driven almost entirely by the bitterness difference:

$$\begin{aligned}d_E(P_1, P_3) &= \sqrt{(110-80)^2 + (220-400)^2} \\ &= \sqrt{900 + 32400} \\ &= \sqrt{33300} \\ &\approx 182.5\end{aligned}$$

The Mahalanobis distance is far smaller because the covariance matrix rescales for the large variance difference between features. The bitterness axis has a much larger spread (σ^2=15600 vs σ^2=3900), so a displacement of 180 bitterness units is less unusual than it appears, and contributes less to the Mahalanobis distance accordingly.

STANDARDIZATION does not change the data, it changes the expanse and spread of the space you measure it in. Units disappear; and make way for a more abstract notion of distance, in terms of how many standard deviations apart the data points are.

$C_1 = (300,\ 7)$
$C_2 = (500,\ 6)$
$C_3 = (700,\ 3)$

daily cups sold, customer rating (1 to 10)

THE EFFECT OF STANDARDIZATION on café data. Three cafés C_1, C_2, C_3 recorded as daily cups sold and customer rating.

Compute the Euclidean distance on raw features, standardize all three points, and recompute. On raw features, daily sales dominate completely; the two pairs involving C_2 look equally close:

$$\begin{aligned} d_E(C_1, C_2) &= \sqrt{(500-300)^2 + (6-7)^2} \\ &= \sqrt{40\,000 + 1} \\ &\approx 200.0 \\ d_E(C_2, C_3) &= \sqrt{(700-500)^2 + (3-6)^2} \\ &= \sqrt{40\,000 + 9} \\ &\approx 200.0 \end{aligned}$$

The rating differences (1 and 9) are drowned out by the cup counts (40,000 each). The means and standard deviations are:

$$\mu_{\text{cups}} = 500, \qquad \sigma_{\text{cups}} = 200$$
$$\mu_{\text{rating}} \approx 5.33, \qquad \sigma_{\text{rating}} \approx 2.08$$

After z-score normalization the cafés map to:

$$C_1' \approx (-1,\ 0.80), \qquad C_2' \approx (0,\ 0.32), \qquad C_3' \approx (1,\ -1.12)$$

THE CUPS standardize cleanly because $300, 500, 700$ are evenly spaced. The ratings do not: $7, 6, 3$ give a non-integer mean and an irrational standard deviation. This is what real data looks like.

Recomputing the distances:

$$\begin{aligned} d(C_1', C_2') &= \sqrt{(0-(-1))^2 + (0.32-0.80)^2} \\ &= \sqrt{1 + 0.23} \\ &\approx 1.11 \\ d(C_2', C_3') &= \sqrt{(1-0)^2 + (-1.12-0.32)^2} \\ &= \sqrt{1 + 2.07} \\ &\approx 1.75 \end{aligned}$$

C_2 is now clearly closer to C_1 than to C_3. In raw space, sales made both pairs look identical; after standardization, the similar reviews of C_1 and C_2 pull them together while C_3's low rating pushes it away.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

20 coffee samples
acidity (pH, 4.0 to 7.0)
bitterness (1 to 10)
brew strength (mg/mL, 5 to 25)

four varieties: cold brew, drip, espresso, latte

A TOY DATASET TO EXPLORE. You are given measurements of 20 coffee samples with three features: *acidity* (pH, 4.0 to 7.0), *bitterness*

(1 to 10), and *brew strength* (mg/mL, 5 to 25). The samples belong to four varieties: cold brew, drip, espresso, and latte. Load the dataset and scatter-plot acidity vs. bitterness. Then compute the Euclidean distance matrix and identify the most similar pair, is the result plausible? Apply z-score normalization, recompute the distance matrix, and observe how the nearest neighbors shift. Compute cosine similarity between all pairs and compare it to the Euclidean grouping. Finally, compute the sample covariance matrix of the normalized data and check whether any features are correlated.

WHAT TO OBSERVE. Without normalization, brew strength dominates similarity because its scale (5 to 25 mg/mL) dwarfs the others. A latte and an espresso with similar strength appear closer than two espressos with the same acidity and bitterness profile. After normalization, all three features contribute equally and the four natural clusters become visible. Cosine similarity groups samples by the *ratio* of their features, which may merge varieties with similar flavor profiles but very different intensities.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during the exercises and afterwards:

- How do we distinguish between supervised and unsupervised learning?
- Elaborate on the differences between $\tilde{y}$, $\hat{y}$, and y?
- What are the key properties that define a distance metric?
- How do different distance metrics capture different notions of similarity?
- What is a feature space, and how do data points relate to it?
- What do we mean by feature engineering and feature transformation?
- What can happen when features are on different scales or magnitudes?
- How can we engineer and transform feature spaces?
- Why do we transform feature spaces with respect to pattern? And what are the common transformations?

RECAP of Key Concepts:

- Unsupervised learning discovers pattern in unlabeled data
- Distance metrics define what similar means
- Feature spaces can be engineered and transformed to reveal different patterns

SO FAR, WE DID MEASURE SIMILARITY BETWEEN SINGLE DATA POINTS. How do we embed this information across data points? Is it possible to represent pattern in the data as a whole, rather than just pairwise similarities? In the next chapter we explore clusters and clustering algorithms, which are the most classic way to represent pattern in unsupervised learning.

WITHOUT KNOWING ABOUT LABELS OR SEMANTICS we can tell which samples are similar.

TEASER. How do we represent structure beyond pairwise similarities and distances?

FEEDBACK

Centroid Clustering

From Similar to of the Same

The Why

In the previous chapter, we established that distance is a choice that is to be engineered[4]: it encodes what similar means in a given feature space. But similarity is a local, pairwise concept so far.

As a human looking at data, we immediately see clusters C of points that belong together, and we have an intuition for which new points would fit into which cluster. The process of finding them is called clustering.

[4] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

A partition or a clustering assigns every point to exactly one cluster, with no cluster left empty. In this lecture we will learn about K-Means[5], which produces a *hard* partition: membership is binary, not graded.

[5] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967. https://scholar.google.com/scholar?q=MacQueen+1967+methods+classification+multivariate+observations

Binary membership is a strong assumption that later clustering methods will soften.

Given that means to encode similarity over whole sets of samples, we can ask a more complex question, one that goes beyond pairs of points. Being able to answer which point belongs to which cluster is a predictive power that allows us to generalize from observed data to new instances.

In order to move from representation to leveraging pattern we have good reason to find ways to,

- compute partitions, of the data, given a distance measure.
- formalize what makes a well-defined partition by specifying an appropriate objective.
- recognize the geometric assumptions and finding ways to represent cluster characteristics.

Figure 10: Clustering allows us to move from a) and b) are similar to a) and b) are of the same. *Image is generated with NanoBanana.*

Hands On Experience

CONSIDER twelve data points in two dimensions. Which points belong together? How many clusters do you see?

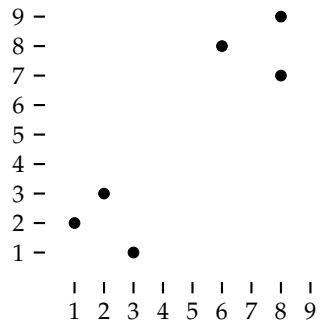


Figure 11: Six points in 2D. How many clusters do you see?

HOW MANY CLUSTERS do you see? Two seems natural, but could you argue for one? For six?

WITHOUT GROUND TRUTH, there is no single right answer, as a context for any answer could be engineered. However, some answers are better than others, and it does not come as a surprise when I tell you that most people see two clusters in this data.

THE FUNDAMENTAL QUESTION this chapter answers is how do we find the clusters mathematically? We will see how K-Means assigns each point to a cluster; and how cluster centroids function as representative points for each cluster.

WHERE WOULD YOU PLACE a representative point for each cluster? Could you think of other measures to characterize and represent clusters, beyond a single point?

A CENTROID c is the mean position of all points assigned to a cluster. It need not be an actual data point; it is the center of mass of the cluster. The centroid exists in the same space as the data. With a different distance, a different notion of center is needed. For sake of simplicity and interpretability, we tie ourselves to Euclidean distance and arithmetic means. But it is worth getting an intuition for other distances as well.

THE LEARNING OBJECTIVES of this lecture:

- Understand the K-Means objective and algorithm, including convergence.
- Apply initialization strategies and K-selection methods.
- Identify when K-Means fails and understand why.

The K-Means Algorithm

A DATA POINT is a vector $x_i \in \mathbb{R}^d$, where d is the number of features and $i = 1, \ldots, n$ indexes the observations.

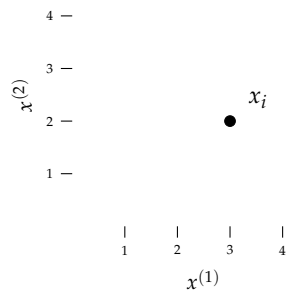


Figure 12: A single data point x_i in $\mathbb{R}^2$.

A CLUSTER is a non-empty subset $C_k \subseteq \{x_1, \ldots, x_n\}$. Each point belongs to exactly one cluster; this is the hard assignment assumption, meaning clusters are disjoint:

$$C_j \cap C_k = \emptyset \quad \text{for } j \neq k$$

A clustering of the data is a collection of K clusters $\{C_1, \ldots, C_K\}$ that is exhaustive, every point is accounted for:

$$\bigcup_{k=1}^{K} C_k = \{x_1, \ldots, x_n\}$$

K-MEANS finds a clustering of n data points into K clusters by minimizing the total squared distance from each point to its cluster's centroid. This quantity is called inertia or within-cluster sum of squares (WCSS):

MINIMIZING J is NP-hard (effort exponential in n and K) in general. K-Means finds a local optimum, not necessarily the global one. The result depends on initialization, and is thus non-deterministic. Shout out numerical methods.

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \tag{7}$$

where the mean $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ is the centroid of cluster C_k.

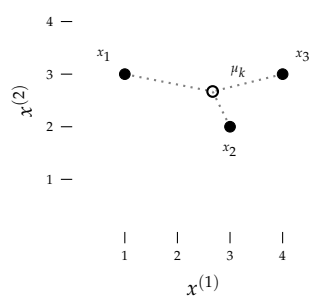


Figure 13: Cluster $C_k = \{x_1, x_2, x_3\}$ with centroid μ_k. Dotted lines show the distances whose squares sum to J.

THE ALGORITHM ALTERNATES BETWEEN TWO STEPS until the assignments no longer change. It remains one of the most cited algorithms in machine learning, in part because it is easy to implement, easy to interpret, and fast enough to run on large datasets.

Algorithm 1: K-Means

Require: Data $X = \{x_1, \ldots, x_n\}$, number of clusters K, initial centroids $\mu_1, \ldots, \mu_K$
Ensure: Clustering $\{C_1, \ldots, C_K\}$, centroids $\{\mu_1, \ldots, \mu_K\}$

```
repeat
    Assignment: c_i ← arg min_k ||x_i − μ_k||^2 for each x_i
    Update: μ_k ← (1/|C_k|) Σ_{x_i : c_i = k} x_i for each k
until assignments no longer change
```

STANDARD ALGORITHMIC FORM was described by Lloyd

S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. DOI: 10.1109/TIT.1982.1056489. https://scholar.google.com/scholar?q=Lloyd+1982+least+squares+quantization+PCM

A TRACE THROUGH THE ALGORITHM on three points with $K{=}2$ makes this concrete. With initial centroids at $\mu_1{=}x_1{=}(1,3)$ and $\mu_2{=}x_3{=}(4,3)$:

- For each point, the assignment step asks: which centroid is closest?
- For each cluster, the update step asks: given the current assignments, where is the new centroid?

STEP 1: INITIALIZATION. We place two centroids at $\mu_1 = (1,4)$ and $\mu_2 = (4,1)$, away from any data point. At this moment the positions of the centroids is arbitrary, and the mean variable is not meaningful.

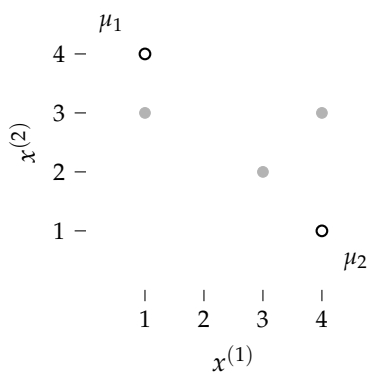


Figure 14: Initialization: two centroids placed away from the data. All points are unassigned.

STEP 2: ASSIGNMENT. Each point joins the cluster of its nearest centroid. For that, we calculate squared distances.

$$
\begin{aligned}
d^2(x_1,\mu_1) &= 0+1 = 1\\
d^2(x_1,\mu_2) &= 9+4\\
&= 13 \;\rightarrow\; C_1\\
d^2(x_2,\mu_1) &= 4+4 = 8\\
d^2(x_2,\mu_2) &= 1+1\\
&= 2 \;\rightarrow\; C_2\\
d^2(x_3,\mu_1) &= 9+1 = 10\\
d^2(x_3,\mu_2) &= 0+4\\
&= 4 \;\rightarrow\; C_2
\end{aligned}
$$

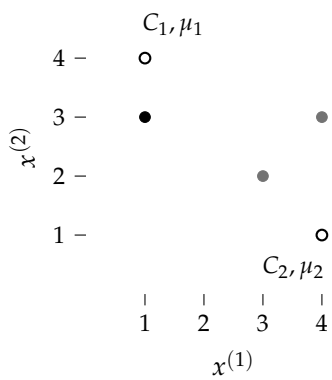


Figure 15: Assignment: $x_1 \rightarrow C_1$ (dark), x_2 and $x_3 \rightarrow C_2$ (grey).

STEP 3: UPDATE. Each centroid moves to the mean of its cluster.

$$
\begin{aligned}
\mu_1' &= x_1\\
&= (1,\ 3)\\
\mu_2' &= \tfrac{1}{2}\big((3,2)+(4,3)\big)\\
&= (3.5,\ 2.5)
\end{aligned}
$$

ON ′ NOTATION. The prime symbol (′) usually denotes updates in iterative algorithms.

CONVERGENCE is guaranteed in finite steps. The argument rests on three observations: first, each assignment step can only decrease

LOCAL OPTIMUM. Convergence does not mean the algorithm found the best clustering. It found a clustering. A different initialization may yield a better one.

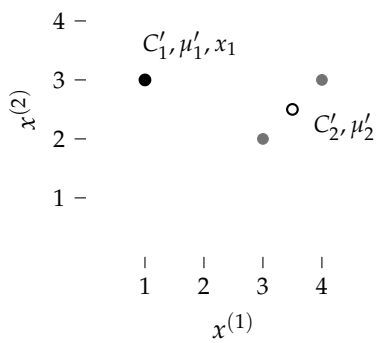


Figure 16: Update: both centroids move. μ_1 coincides with $x_1 = (1, 3)$; μ_2 jumps from $(4, 1)$ to $(3.5, 2.5)$. Rerunning assignment yields the same clusters: converged at J=1.

J, since every point moves to an equal or closer centroid; second, each update step can only decrease J, since the mean minimizes squared distance to its members; third, there are at most K^n possible assignment configurations. Since J is non-increasing and the number of states is finite, the algorithm must reach a fixed assignment and stop.

Initialization Matters

K-Means is sensitive to where centroids start. The algorithm is thus non-deterministic. This is because the objective is non-convex: it has many local minima, and the algorithm may get stuck in one of them. If two initial centroids fall inside the same natural cluster, this cluster might get split. While at the same time, a different cluster might be left without a centroid, getting merged with a nearby cluster instead, which results in a high-inertia J clustering.

Educated InitializationK-Means++ for instance features educated initialization. Centroids seeded onto data points x_i. The first centroid is chosen uniformly at random. Each subsequent centroid is chosen with probability proportional to its squared distance to the nearest already-chosen centroid: $p(x_i) = D(x_i)^2 / \sum_j D(x_j)^2$, where $D(x_i) = \min_{j<k} \|x_i - c_j\|$. Points far from all current centroids are more likely to be chosen next. This gives a theoretical guarantee on the expected inertia relative to the global optimum.

D. Arthur and S. Vassilvitskii. K-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007. https://scholar.google.com/scholar?q=Arthur+Vassilvitskii+2007+k-means+advantages+careful+seeding

Running the same three points with μ_1=$(2, 2)$ and μ_2=$(4, 4)$ shows how a different start leads to a different, worse result.

Multiple restarts. Running K-Means n times with different random initializations, keep the result with the lowest J. Typical default: n=10, but checking the variance or standard deviation of the resulting Inertias and making an informed decision is recommended.

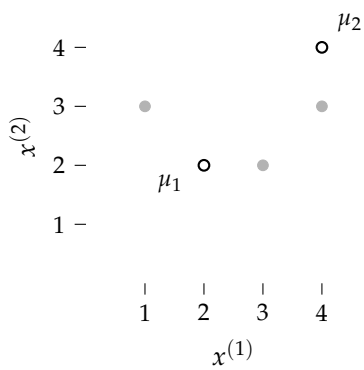


Figure 17: Initialization: centroids at μ_1=$(2,2)$ and μ_2=$(4,4)$. All points unassigned.

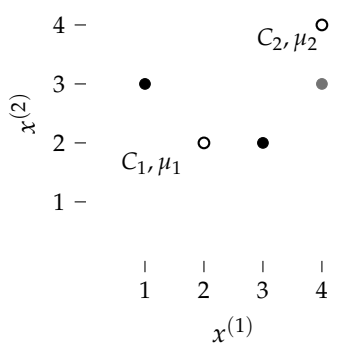


Figure 18: Assignment: x_1 and $x_2 \to C_1$ (dark), $x_3 \to C_2$ (grey).

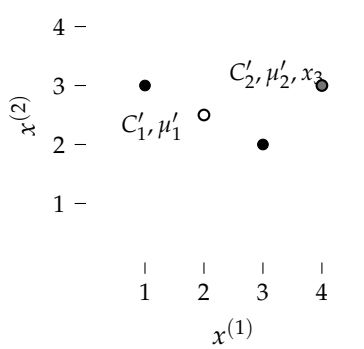


Figure 19: Update: μ_1 moves to $(2, 2.5)$, μ_2 drops to $(4, 3)$. Re-running assignment yields the same clusters: converged at J=2.5, worse than the earlier trace (J=1).

Choosing K

In 2D the number of clusters is often obvious, but in higher dimensions and with more data, or when facing fuzzy clusters or cluster borders, it is not. While the choice for K is often guided by domain knowledge, it is not uncommon to have no such guidance. Then we have to rely on heuristics.

The elbow method plots inertia J against K and looks for a kink: the point where adding one more cluster yields diminishing returns. For our three points, J drops from K=1 to K=2, then reaches zero at K=3.

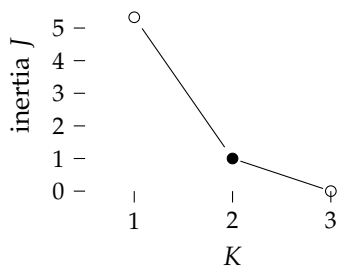


Figure 20: Inertia J as a function of K for the three trace points. The kink at K=2 (filled dot) matches the two natural clusters. K=3 trivially gives J=0 - jap, that is overfitting.

The elbow is not always clear. On real data, the curve may decrease smoothly with no obvious kink. The elbow is a heuristic, not a guarantee.

The silhouette score builds on two distances defined for each point x_i. This allows us to evaluate the clustering on a per-point scale. Think about how interpretable and beautiful that would look when colour-coded.

- **Intra-cluster distance** $a(i)$: mean distance from x_i to all other points in its own cluster. Small $a(i)$ means the point fits tightly with its neighbours.
- **Inter-cluster distance** $b(i)$: mean distance from x_i to all points in the nearest other cluster. Large $b(i)$ means the point is far from the next-best alternative.

A well-placed point has small $a(i)$ and large $b(i)$. The silhouette score combines both into a single value per point:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}$$

$s(i)$ ranges from -1; inter $<$ intra: point fits better in a neighbouring cluster. To $+1$; intra $\ll$ inter: point is tightly placed and well-separated The mean silhouette over all points is a quality score for the full clustering. Choose K that maximizes it.

Simplifications in per-cluster metrics. In practice, $a(i)$ is often computed as the distance to the cluster centroid, rather than the mean distance to all other points. And b(i) is often computed as the distance to the nearest other centroid, rather than the mean distance to all points in the nearest cluster. And even more simplifications are possible, when only calculating the silhouette for the centroids themselves.

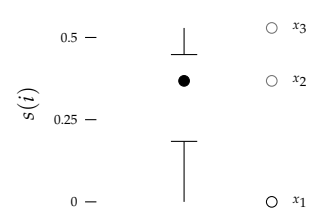
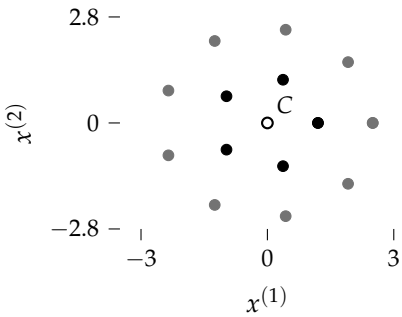


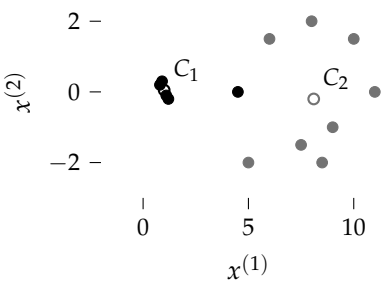
Figure 21: Silhouette scores for K=2. Box plot (left): median ≈0.37, mean ≈0.30, IQR [0.18, 0.45]. Individual scores per point (right, coloured by cluster).

When K-Means Fails

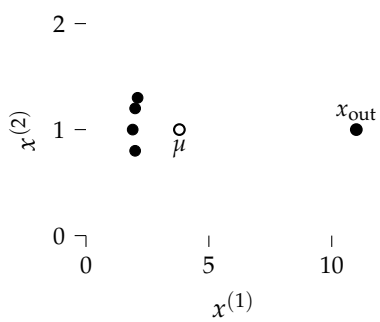
K-Means assumes clusters are roughly spherical, equally sized, and equally dense. Remember that distances measures are a choice.



Non-Convex Shapes
Figure 22: The natural clusters are concentric rings. Both share the same centroid at the origin, so K-Means cannot separate them; it will split the data along a straight boundary through the centre instead.



Cluster Density and Gravity
Figure 23: C_1 has four tightly packed points, C_2 has eight spread over a large area. K-Means minimises total inertia, so the decision boundary is pulled toward the small cluster, risking misassignment of nearby points.



Outliers
Figure 24: Sensitivity to outliers: four tight points form a natural cluster near $(2, 1)$, but a single outlier ($\times$) pulls the centroid μ to $(3.8, 1)$, well outside the dense group. K-Means uses the mean, which is not robust to extreme points.

When K-Means struggles, the underlying issue is always the same: the Euclidean distance to a centroid is the wrong measure of belonging for that data. Can you think of another failure mode? Let me know.

K-Medoids uses actual data points as cluster centers instead of means, making it robust to outliers and compatible with non-Euclidean distances.

Mini-Batch K-Means updates centroids on random subsets of the data only, allowing us to scale to millions of points at the cost of a slightly noisier result.

Examples & Exercises

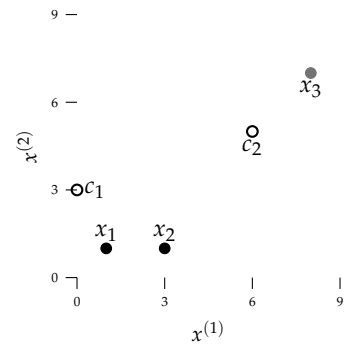


Figure 25: Three coffee samples x_1, x_2, x_3 with initial centroids $\mu_1{=}(0,3)$ and $\mu_2{=}(6,5)$ (circles).

COMPUTING BY HAND builds the intuition that no amount of function calls can replace. Step through the arithmetic slowly. Implementing from scratch comes right after, but let's anchor the mechanics first.

GIVEN THREE POINTS $x_1{=}(1,1)$, $x_2{=}(3,1)$, $x_3{=}(8,7)$, run one full K-Means iteration with $K{=}2$. The initial centroids are $c_1{=}(0,3)$ and $c_2{=}(6,5)$, neither coinciding with a data point. Your tasks: perform the assignment step, the update step, verify convergence, and evaluate the quality of the clustering by computing the inertia.

ASSIGNMENT STEP. For each point compute $d^2(x, c_k) = \sum_j (x_j - c_{kj})^2$ and assign to the nearest centroid. Here is the full trace for point $x_1{=}(1,1)$:

$$\begin{aligned} d^2(x_1,\, c_1) &= (1-0)^2 + (1-3)^2 \\ &= 1 + 4 \\ &= 5 \end{aligned}$$

$$\begin{aligned} d^2(x_1,\, c_2) &= (1-6)^2 + (1-5)^2 \\ &= 25 + 16 \\ &= 41 \end{aligned}$$

Since $5 < 41$, point x_1 is assigned to C_1. Now compute the distances for x_2 and x_3 yourself, then verify against the table below.

Point	$d^2(\cdot, \mu_1)$	$d^2(\cdot, \mu_2)$	**Cluster**
x_1 $(1,1)$	**5**	41	C_1
x_2 $(3,1)$	**13**	25	C_1
x_3 $(8,7)$	80	**8**	C_2

Table 1: Assignment step with $c_1{=}(0,3)$, $c_2{=}(6,5)$. Bold values indicate the minimum in each row; each point is assigned to the corresponding cluster.

UPDATE STEP. Recompute each centroid as the mean of its assigned points. Here is the full calculation for μ_1':

$$\begin{aligned}\mu_1' &= \frac{1}{|C_1|}\sum_{x\in C_1} x \\ &= \tfrac{1}{2}\big((1,1)+(3,1)\big) \\ &= \tfrac{1}{2}(4,\,2) \\ &= (2,\,1)\end{aligned}$$

Now derive μ_2' yourself (C_2 contains only a single point) and then check:

$$\begin{aligned}\mu_2' &= x_3 \\ &= (8,\,7)\end{aligned}$$

CONVERGENCE CHECK. Run the assignment step once more with $\mu_1'{=}(2,1)$ and $\mu_2'{=}(8,7)$: you should recover the same two clusters $C_1{=}\{x_1,x_2\}$, $C_2{=}\{x_3\}$. The algorithm has converged after a single iteration.

The inertia at convergence is:

$$\begin{aligned}J &= \|x_1-\mu_1'\|^2+\|x_2-\mu_1'\|^2+\|x_3-\mu_2'\|^2 \\ &= \big((1{-}2)^2+(1{-}1)^2\big)+\big((3{-}2)^2+(1{-}1)^2\big)+0 \\ &= 1+1+0 \\ &= 2\end{aligned}$$

Calculate the inertia for $K = 1$ yourself. Assume that the single centroid is at the mean of all three points, and is thus quick to compute. What is the Inertia for $K = 3$, is there a need to compute it, and why?

CHOOSING K WITH THE ELBOW METHOD. The table in the margin gives the optimal inertia for $K = 1, 2, 3$ on the same three points. Sketch the inertia curve and identify the elbow: the value of K at which adding another cluster stops yielding a large reduction. Which K do you select, and why?
Note that $K{=}3$ always reaches $J{=}0$ by placing one centroid per point; this tells us nothing about structure in the data.

Inertia values:

K	J
1	50
2	2
3	0

COMPUTE THE SILHOUETTE SCORES (optional) for the three trace points $x_1{=}(1,3)$, $x_2{=}(3,2)$, $x_3{=}(4,3)$ with $K{=}2$, where $C_1{=}\{x_1\}$ and $C_2{=}\{x_2, x_3\}$.

For x_1 (singleton cluster, so $a(1){=}0$ by convention):

$$\begin{aligned} b(1) &= \tfrac{1}{2}\big(d(x_1,x_2) + d(x_1,x_3)\big) \\ &= \tfrac{1}{2}\big(\sqrt{5} + \sqrt{9}\big) \\ &= \tfrac{1}{2}(2.24 + 3) \\ &\approx 2.62 \\ s(1) &= \frac{2.62 - 0}{\max(0,\, 2.62)} \\ &= 0 \end{aligned}$$

For x_2 and x_3 (both in C_2, with $d(x_2,x_3) = \sqrt{2} \approx 1.41$):

$$\begin{aligned} a(2) &= d(x_2,x_3) \approx 1.41 \\ b(2) &= d(x_2,x_1) \\ &= \sqrt{5} \\ &\approx 2.24 \\ s(2) &= \frac{2.24 - 1.41}{2.24} \approx 0.37 \end{aligned}$$

$$\begin{aligned} a(3) &= d(x_3,x_2) \approx 1.41 \\ b(3) &= d(x_3,x_1) \\ &= \sqrt{9} \\ &= 3 \\ s(3) &= \frac{3 - 1.41}{3} \approx 0.53 \end{aligned}$$

The mean silhouette score is $\bar{s} = (0 + 0.37 + 0.53)/3 \approx 0.30$, matching the box plot in the theory section. Note that x_1 drags the mean down: singleton clusters always score $s{=}0$.

A COFFEE AROMA DATASET TO EXPLORE. Load the dataset of 1,797 two-dimensional aroma fingerprints from ten brewing methods and scatter-plot the two aroma coordinates, coloring each sample by its true variety. Implement the assignment step, the update step, and the inertia function from scratch, then combine them into a complete K-Means run. Run K-Means with $K{=}10$ twenty times using different random seeds and compare the resulting inertia values.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

Then vary K from 1 to 15, record the best inertia across three restarts for each, and plot the elbow curve.

RANDOM SEEDS. Random seeds are integers that initialize the pseudo-random number generator, ensuring reproducibility.

WHAT TO OBSERVE. Inertia varies noticeably across the twenty runs at $K{=}10$: some seeds place two starting centroids inside the same

brewing group and the algorithm never recovers, producing a measurably worse partition. The elbow curve bends near $K=10$, though not sharply, because several brewing varieties overlap in the two-dimensional fingerprint space. Adding clusters beyond the elbow yields diminishing inertia reduction, confirming that the algorithm is splitting existing groups rather than discovering new ones.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the K-Means objective actually minimize, and why is that a reasonable definition of a good clustering?
- Why does K-Means always converge, and why does convergence not guarantee the best solution?
- Which two improvements of K-means did we mention around initialization?
- How can we mitigate the effect of outliers on K-Means?
- When would you use the elbow method? When would you not?
- How does the silhouette score measure something different from inertia?
- What are the three most notorious failures of K-Means, and as a teaser how can we mitigate those?
- Given a new dataset, what would be your first three steps before running K-Means?

RECAP of Key Concepts:

- K-Means minimizes inertia by alternating assignment (points to nearest centroid) and update (centroid to cluster mean)
- Multiple random initializations is a common heuristics to mitigate local minima
- The elbow and silhouette score are complementary heuristics for choosing K
- K-Means assumes spherical, equally sized clusters; it fails on rings, elongated shapes, and density differences

K-MEANS IS FLAWED. Every failure mode in this chapter reduces to the same problem; Euclidean distance to a centroid is the wrong measure of membership for that data. But there is a more structural limitation: K-Means must commit to a single K. With K=3 you see roast regions; with K=10 you see brewing methods. You cannot see both at once, and the elbow heuristic only helps you pick one level and drop the remaining pattern.

TEASER How can we built the full hierarchy of clusterings first, capture the complete pattern across all levels, and allow for post hoc selections?

Hierarchical Clustering

All scales at once.

The Why

K-Means must commit to one value of K before seeing the data. With K=2 you recover two broad groups; with K=10 you recover ten fine subgroups. You cannot see both at once, and the elbow heuristic only helps you pick one level and discard the rest.

Hierarchy is the natural answer. Structure in real data rarely lives at a single scale. Biological taxonomy nests species inside genera inside families inside orders. A document corpus organizes words into topics and topics into domains. File systems nest folders inside folders. In each case, the right question is not "how many clusters?" but "at which scale do I look?"

Hierarchical clustering produces a **dendrogram**: a binary tree in which leaves are individual data points and each internal node records the distance at which two sub-trees were merged. A horizontal cut at any chosen height yields a flat partition. The full range of partitions, from K=1 to K=n, lives in this one structure.

The same distance matrix that K-Means uses to assign points to centroids can be used to build an entire tree of clusterings. The tree is built in one pass; every value of K is then available for free.

Hands-On Experience

Consider the same six points from the previous chapter.

Which two points are closest to each other? Connect them with a line. Now find the next closest pair and connect them. Keep going until all six points are linked into a tree. At what level of distance would you cut that tree to recover two groups?

The Learning Objectives of this lecture:

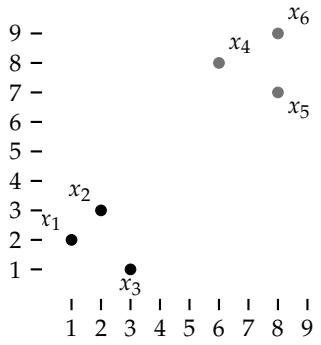


Figure 26: The canonical six points. Which two are closest? Start connecting them one by one, nearest pair first.

- Understand how agglomerative clustering builds a dendrogram through successive minimum-distance merges.
- Apply single, complete, and Ward's linkage and explain how each criterion shapes the resulting hierarchy.
- Identify when hierarchical clustering succeeds where K-Means fails and recognise its own failure modes.

The Hierarchical Clustering Algorithm

THE AGGLOMERATIVE STRATEGY is bottom-up: start with n singleton clusters, one per point, and greedily merge the two closest clusters at each step. Every merge is recorded as an internal node labeled with the distance at which it occurred.

Algorithm 2: Agglomerative Hierarchical Clustering

Require: Data $X = \{x_1, \ldots, x_n\}$, linkage criterion L
Ensure: Dendrogram T
Initialize: $C_i \leftarrow \{x_i\}$ for $i = 1, \ldots, n$; active $\leftarrow \{C_1, \ldots, C_n\}$
Compute $D[i, j] \leftarrow d(x_i, x_j)$ for all $i \neq j$
while $|\text{active}| > 1$ **do**
  $(i^*,\ j^*) \leftarrow \arg\min_{i \neq j}\ D[i, j]$
  Record merge: $(C_{i^*},\ C_{j^*})$ at height $D[i^*, j^*]$ in T
  $C_{\text{new}} \leftarrow C_{i^*} \cup C_{j^*}$
  Update $D[\text{new}, k] \leftarrow L(C_{\text{new}},\ C_k)$ for each active $C_k \neq C_{i^*}, C_{j^*}$
  Remove C_{i^*} and C_{j^*} from active; add C_{new}
end while

THE RESULT is a binary tree. Every valid flat partition from $K{=}1$ to $K{=}n$ can be read off by making a horizontal cut at the desired height. The $n - 1$ merge heights span the full range from the smallest pairwise distance to the diameter of the data.

DIVISIVE CLUSTERING (top-down) starts with all points in one cluster and recursively splits the cluster with the largest diameter. It is rarely used in practice: splitting is more expensive than merging, and early splits made with global information can be noisier than local merges.

LINKAGE CRITERIA determine how the distance between two clusters is computed from the distances between their individual members. The choice of linkage profoundly shapes the dendrogram.

Single Linkage

$$D_{\text{single}}(C_i,\ C_j) = \min_{x \in C_i,\ y \in C_j} d(x,\ y)$$

The distance between two clusters is the distance between their nearest members. Single linkage is fast and can detect elongated, non-convex clusters. Its notorious weakness is **chaining**: a single close pair of points can bridge two otherwise distant clusters, pulling them into one.

Complete Linkage

$$D_{\text{complete}}(C_i, C_j) = \max_{x \in C_i,\, y \in C_j} d(x, y)$$

The distance between two clusters is the distance between their furthest members. Complete linkage tends to produce compact, roughly equal-diameter clusters and is robust to chaining. It may, however, split naturally elongated shapes.

Average Linkage

$$D_{\text{average}}(C_i, C_j) = \frac{1}{|C_i|\,|C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

Average linkage averages all cross-cluster pairwise distances. It is a middle ground between single and complete linkage.

UPGMA. Average linkage is also known as the Unweighted Pair Group Method with Arithmetic Mean. It is a compromise: less prone to chaining than single linkage, less aggressive at squashing elongated clusters than complete linkage.

Ward's Method

Ward's criterion merges the two clusters whose union minimises the increase in total within-cluster variance:

$$D_{\text{Ward}}(C_i, C_j) = \frac{|C_i|\,|C_j|}{|C_i| + |C_j|} \left\|\mu_i - \mu_j\right\|^2$$

where μ_i and μ_j are the centroids of C_i and C_j. Ward's method produces compact, spherical clusters and is closely related to K-Means: both minimise within-cluster variance.

ONLY EUCLIDEAN. Ward's formula requires Euclidean distances and cluster centroids. Single, complete, and average linkage work with any distance or dissimilarity matrix, including non-Euclidean ones.

THE DENDROGRAM for the canonical six points under single linkage is shown below. Reading it from bottom to top: leaves are the six data points; horizontal bars mark merges; the height of each bar is the distance at which that merge occurred.

READING A DENDROGRAM requires only one rule: a horizontal cut at height h partitions the data into as many clusters as there are vertical stems crossing that line. Cutting above 6.40 gives one cluster; cutting between 2.24 and 6.40 gives two; cutting between 2.00 and 2.24 gives three; cutting just above zero gives n singletons. A large gap between consecutive merge heights is the dendrogram equivalent of the elbow: it suggests a natural number of clusters.

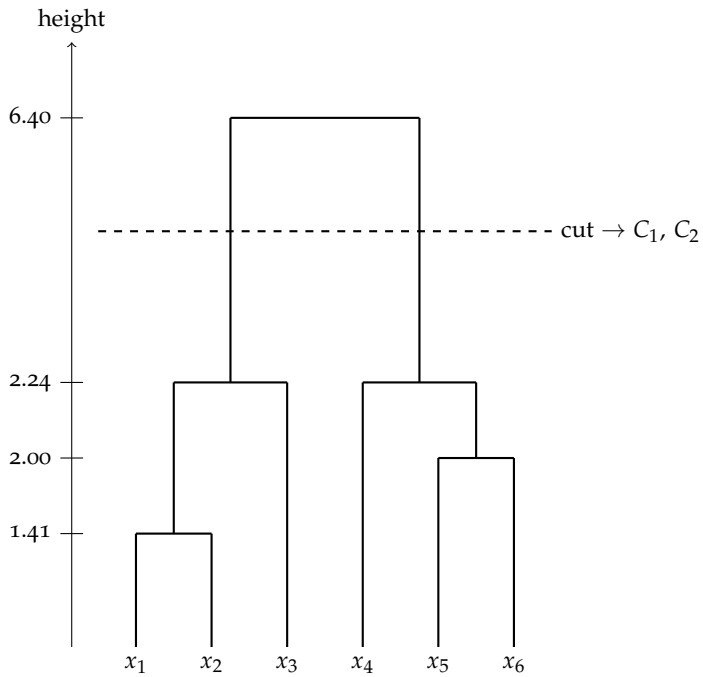
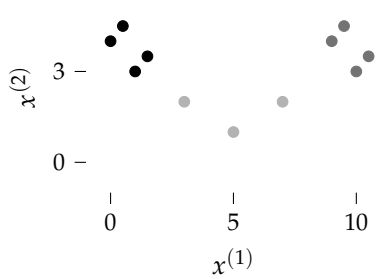


Figure 27: Single-linkage dendrogram for $x_1, \ldots, x_6$. Merge heights are Euclidean distances. Steps 3 and 4 both occur at height $\sqrt{5}\approx 2.24$. The large gap between 2.24 and 6.40 signals that cutting here yields the two natural clusters $C_1 = \{x_1, x_2, x_3\}$ and $C_2 = \{x_4, x_5, x_6\}$.

Failure Modes of Hierarchical Clustering

HIERARCHICAL CLUSTERING HAS ITS OWN PATHOLOGIES. Understanding them is essential before choosing a linkage criterion or trusting a dendrogram.



CHAINING

Figure 28: Single-linkage chaining. Three bridge points (light grey) connect two otherwise separate dense clusters. Single linkage merges all nine points into one cluster; complete or Ward's linkage keeps the two groups separate because no individual merge is small enough to bridge the full cluster diameter.

OUTLIER SENSITIVITY

WHEN HIERARCHICAL CLUSTERING STRUGGLES, the shared root cause is that every point, no matter how isolated, must eventually merge into a cluster. There is no concept of noise: outliers and bridge points create spurious merges that distort the dendrogram and make merge heights unreliable as signals of natural structure.

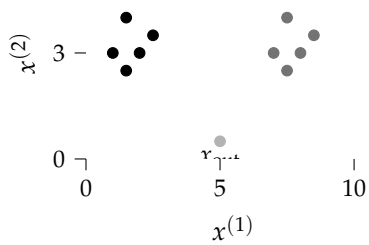


Figure 29: Outlier sensitivity. The outlier x_{out} sits between the two clusters. It is absorbed into one of them at an intermediate merge height, inflating that merge height and obscuring the true gap between the two clusters in the dendrogram.

Computational Complexity

Algorithm	Time	Space	Notes
Naive agglomerative	$O(n^3)$	$O(n^2)$	Recomputes all distances each step
Single linkage (SLINK)	$O(n^2)$	$O(n)$	Optimal algorithm for single linkage
Complete / average linkage	$O(n^2 \log n)$	$O(n^2)$	Priority-queue acceleration
BIRCH (approximate)	$O(n)$	$O(n)$	Approximate; clusters a CF-tree

Table 2: Computational complexity of hierarchical clustering variants. Naive agglomerative is $O(n^3)$ because it rescans the full distance matrix at each of the $n-1$ merge steps.

SCALABILITY. For $n > 10{,}000$ consider BIRCH, which builds a compact CF-tree over the data first and then applies hierarchical clustering to the tree nodes rather than to individual points.

THE QUADRATIC DISTANCE MATRIX is the fundamental bottleneck. Storing all $\frac{n(n-1)}{2}$ pairwise distances requires $O(n^2)$ memory. For n=10,000 that is already 400 million floats; for n=100,000 it is infeasible without approximation.

Examples & Exercises

COMPUTING BY HAND builds the intuition that no library call can replace. Step through each merge slowly; feel when the algorithm connects the right pair and when it is about to build a chain.

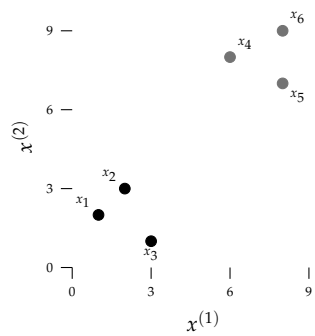


Figure 30: Six points for the by-hand trace.

GIVEN SIX POINTS x_1=(1,2), x_2=(2,3), x_3=(3,1), x_4=(6,8), x_5=(8,7), x_6=(8,9), run single-linkage agglomerative clustering step by step. Your tasks: fill in the initial pairwise distance table, identify the minimum pair at each step, record each merge, update distances using single linkage, and construct the dendrogram.

INITIAL DISTANCES. For each pair (x_i, x_j) compute $d(x_i, x_j) = \sqrt{\sum_k (x_i^{(k)} - x_j^{(k)})^2}$. Here is the full calculation for $d(x_1, x_2)$:

$$\begin{aligned} d(x_1, x_2) &= \sqrt{(2-1)^2 + (3-2)^2} \\ &= \sqrt{1+1} \\ &= \sqrt{2} \\ &\approx 1.41 \end{aligned}$$

Compute the remaining distances yourself, then verify against the table below.

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	–	**1.41**	2.24	7.81	8.60	9.90
x_2	1.41	–	2.24	6.40	7.21	8.49
x_3	2.24	2.24	–	7.62	7.81	9.43
x_4	7.81	6.40	7.62	–	2.24	2.24
x_5	8.60	7.21	7.81	2.24	–	**2.00**
x_6	9.90	8.49	9.43	2.24	2.00	–

Table 3: Initial pairwise Euclidean distances. The global minimum $d(x_1, x_2)$=1.41 (bold) triggers the first merge. The next smallest, $d(x_5, x_6)$=2.00 (bold), triggers the second.

FIRST MERGE. The minimum entry is $d(x_1, x_2)$=1.41. Merge x_1 and x_2 into cluster $\{x_1, x_2\}$ at height 1.41. Update distances using single linkage: replace the two rows and columns for x_1 and x_2 with one new row and column, taking the minimum of the two old distances:

$$
\begin{aligned}
d(\{x_1, x_2\},\ x_3) &= \min(d(x_1, x_3),\ d(x_2, x_3)) \\
&= \min(2.24,\ 2.24) \\
&= 2.24 \\
d(\{x_1, x_2\},\ x_4) &= \min(d(x_1, x_4),\ d(x_2, x_4)) \\
&= \min(7.81,\ 6.40) \\
&= 6.40
\end{aligned}
$$

Derive the updates for $\{x_1, x_2\}$ vs. x_5 and x_6 yourself, then trace the remaining merges and verify against the solution table.

Step	Clusters merged	Height	Updated distances (single linkage)
1	$x_1,\ x_2$	1.41	$d(\{x_1,x_2\},x_3)$=2.24; $d(\{x_1,x_2\},x_4)$=6.40; $d(\{x_1,x_2\},x_5)$=7.21; $d(\{x_1,x_2\},x_6)$=8.49
2	$x_5,\ x_6$	2.00	$d(x_4,\{x_5,x_6\})$=2.24; $d(\{x_1,x_2\},\{x_5,x_6\})$=7.21; $d(x_3,\{x_5,x_6\})$=7.81
3	$\{x_1,x_2\},\ x_3$	2.24	$d(\{x_1,x_2,x_3\},x_4)$=6.40; $d(\{x_1,x_2,x_3\},\{x_5,x_6\})$=7.21
4	$x_4,\ \{x_5,x_6\}$	2.24	$d(\{x_1,x_2,x_3\},\{x_4,x_5,x_6\})$=6.40
5	$\{x_1,x_2,x_3\},\ \{x_4,x_5,x_6\}$	6.40	Final merge; algorithm complete

Table 4: Single-linkage merge history for $x_1,\ldots,x_6$. Steps 3 and 4 both occur at height $\sqrt{5}\approx 2.24$; ties are broken arbitrarily. The large gap between 2.24 and 6.40 confirms the two natural clusters.

Convergence check. Using the merge history, sketch the dendrogram on paper. Mark the horizontal cut that recovers exactly two clusters. Why does the large gap between heights 2.24 and 6.40 make this cut easy to identify, and what would happen if the two natural clusters were less well separated?

Linkage comparison. Repeat steps 1 and 2 using complete linkage: replace each min in the update rule with max.

Note that for this dataset the merge order does not change: the two natural clusters are so well separated that no linkage criterion alters the dendrogram topology. Now repeat on a dataset with a bridge point added at (4, 4) and observe how single and complete linkage diverge.

Complete-linkage update for step 1:

$d_{\text{comp}}(\{x_1,x_2\},x_3) = \max(2.24,\ 2.24) = 2.24$

$d_{\text{comp}}(\{x_1,x_2\},x_4) = \max(7.81,\ 6.40) = 7.81$

Does the second merge still happen at height 2.00 under complete linkage?

Cophenetic correlation (optional). The cophenetic distance $c(i,j)$ is the height at which x_i and x_j first share a common ancestor in the dendrogram. Read $c(x_1,x_3)$, $c(x_1,x_4)$, and $c(x_2,x_5)$ from the merge table. Compute the Pearson correlation between the three original Euclidean distances and the corresponding cophenetic distances. A value near 1 confirms the dendrogram faithfully preserves the distance structure.

A city-climate dataset to explore. Load the dataset of 50 European cities with two climate features: mean July temperature and mean annual precipitation. Apply single-linkage, complete-linkage,

Code will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

and Ward's linkage hierarchical clustering and plot all three dendrograms side by side.

Cut each dendrogram at $K=3$ and compare which cities fall into the same cluster under each linkage. Compute the cophenetic correlation for all three to quantify which linkage best preserves the original distance structure.

WHAT TO OBSERVE. Single linkage typically produces one large chain-cluster and several singletons for this data; Ward's and complete linkage yield more balanced partitions. The cophenetic correlation is highest for average or complete linkage on most Euclidean datasets, confirming the visual impression of faithfulness. Notice also which cities appear as singletons under single linkage and whether those same cities appear as outliers in the scatter plot.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the merge height in a dendrogram represent, and why does a large gap between consecutive heights signal a natural number of clusters?
- Why does single linkage produce chaining while complete linkage does not? What property of each formula causes this difference?
- In what situations would you prefer Ward's linkage over single or complete linkage? When would Ward's fail?
- K-Means assigns every point to a cluster and never leaves any unassigned. How does hierarchical clustering behave with respect to outliers, and how is this different from how you would handle them in practice?
- You compute a dendrogram and find that the top-level merge height is only marginally larger than the second-to-top merge height. What does this tell you about the cluster structure?
- How does the choice of linkage criterion interact with the choice of distance metric? Can you think of a combination that would yield meaningless results?
- What is the computational bottleneck of hierarchical clustering, and what approximate methods exist to overcome it at scale?

- A colleague argues that hierarchical clustering is always better than K-Means because it does not require specifying K upfront. Under what conditions is this argument wrong?

RECAP of Key Concepts:

- Agglomerative clustering greedily merges the two closest clusters at each step, producing a dendrogram that encodes all flat partitions from $K{=}1$ to $K{=}n$ in a single structure
- Single linkage detects elongated clusters but suffers from chaining; complete and Ward's linkage produce compact clusters and are more robust to sparse bridge points
- A large gap between consecutive merge heights is the dendrogram signal for a natural number of clusters, analogous to the elbow in K-Means inertia curves
- The $O(n^2)$ distance matrix and $O(n^2)$–$O(n^3)$ merge loop limit hierarchical clustering to moderate-sized datasets without approximation methods such as BIRCH
- Outliers and bridge points are never labelled as noise: they are absorbed into some cluster at a finite merge height, distorting the dendrogram

THE ROOT OF HIERARCHICAL FAILURE. Every linkage criterion measures distance between points or boundaries. No criterion has a concept of empty space or of a region too sparse to be a cluster.

HIERARCHICAL CLUSTERING HAS NO NOTION OF EMPTY SPACE. No matter how isolated a point is from every other point, it will be absorbed into a cluster at the last merge. A long, isolated branch in the dendrogram means "last to join," not "does not belong." The chaining failure reveals the same blindspot from the other side: a sparse bridge of points is enough to merge two genuinely separate clusters, because the algorithm sees only pairwise distances, not the density of the surrounding region. With $K{=}2$ you might recover the two natural groups, but an outlier joins one of them silently, without warning and without any measure of how ill-fitting that assignment is.

DENSITY-BASED METHODS redefine cluster membership entirely: a cluster is a maximal region of space that is dense enough, and any point in a sparse region is declared noise outright. This eliminates chaining and handles outliers naturally, without any choice of K or linkage criterion. The next chapter derives DBSCAN, the canonical density-based algorithm, starting from the single question that hierarchical clustering cannot answer: what does it mean for a region to be empty?

Index

agglomerative clustering, 43
average linkage, 44

BIRCH, 46

Centroid Clustering, 26
centroid clustering, 27
  limitations, 41
chaining, 43
cluster, 29
clustering, 29
complete linkage, 44
computational complexity
  hierarchical clustering, 46
Cosine similarity, 17
covariance matrix, 18

data point, 29
data preprocessing, 19
DBSCAN, 51
dendrogram, 41
  reading, 44
distance, 13
distance metric, 16
Distance Metrics, 13
divisive clustering, 43

elbow method, 33
Euclidean distance, 16
examples, 21, 35, 47
exercises, 21, 35, 47

failure modes, 34
  hierarchical clustering, 45
feature space, 16
Foundations, 13

hierarchical clustering, 41

inertia, 29

K-Means, 26, 27, 29
K-Means++, 31

L1 distance, 17
license, 2
linkage criterion, 43

Mahalanobis distance, 18
Manhattan distance, 17
Min-Max scaling, 20

normalization, 19

recap, 25, 38, 50
reflection, 25, 38, 50

silhouette score, 33
similarity, 13
single linkage, 43
standardization, 19

unsupervised learning, 13

Ward's method, 44

Z-score normalization, 19